

ACTUALIS STACK

Architecture Vision and
Implementation Blueprint

Systems shaped by reality.

A reality-first foundation for governed manufacturing and education operations

Version 0.1 | 17 July 2026 | Architecture working paper

Working name only - trademark and domain clearance required before public use.

PART 00

Executive decision

Needs and operational reality dictate the technology - never the reverse.

This paper recommends building Actualis Stack: a governed decision-and-execution foundation for constraint-intensive operations. It is designed first for manufacturing and education, where people, machines, time, qualifications, policies, commitments, exceptions and evidence collide. It can support an MES, APS, task scheduler, education operations product, commerce experience or public site without turning their semantics into one generic CRUD model.

The durable architecture is not Elixir, Rust, React or PostgreSQL. It is the set of boundaries that lets those technologies be replaced: governed capabilities; domain-owned schemas and invariants; independently versioned experience packages; explicit data classes; cell isolation; observation-to-decision provenance; and AI treated as a constrained principal rather than an authority.

The first architecture decision

Decision area	Initial decision	Why it survives change
Product center	Governed capabilities, commitments, decisions, execution and evidence	Avoids DocType, module and solver becoming the universal abstraction
Shape	Capability-centric modular monolith; extract only isolated workloads	Preserves transactions and developer speed while keeping deliberate seams
Experience	Headless core plus independent, signed experience packages	The same domain can support radically different interaction models
Data	A store owner for each data class; PostgreSQL for relational truth	Uses fit-for-purpose storage without pretending all databases are interchangeable
Scale	Cell architecture, workload lanes and bounded resource pools	Limits blast radius and prevents telemetry, AI or solvers from starving users
Delivery	Immutable artifact promotion across disposable sandboxes, test, pre-production and production	Makes rapid deployment reproducible while preserving migration, security and audit evidence
Integration	Wire protocols are adapters around stable capability and event contracts	Webhooks, JSON-RPC and future transports can change without changing domain authority
Machines	Edge gateway, separate observation and command paths	Handles offline plants and device trust without making the cloud a PLC
Permissions	Capability + relationship + attributes + purpose + context + risk	Makes precise authorization explainable, testable and consistent across every surface and principal
AI	Build-time copilot plus runtime bounded principal	Model providers can change; authority, tests and evidence do not

The implementation hypothesis

Start with Elixir/Phoenix for coordination, realtime and governed APIs; PostgreSQL for authoritative relational state; TypeScript/React for the official experience SDK; local SQLite/OPFS, service workers and Web Workers for responsive offline clients; Rust for edge agents, WebAssembly and decision orchestration; established solvers for optimization; object storage for evidence; and ClickHouse as the default candidate for high-volume telemetry after benchmark validation. Begin with a PostgreSQL outbox and durable jobs; add NATS, Temporal, OpenSearch or extra stores only when a measured workload requires them.

OUTCOME

Build one walking skeleton and three proof slices before calling this a platform. The architecture earns its abstractions through observed reuse.

PART 01

How to read this paper

This is a decision document, not a vendor selection brochure. It records what must remain true, what we are choosing now, what is deliberately deferred, and how each claim will be tested.

Document control

Field	Value
Status	Architecture vision - proposed baseline
Version	0.1
Decision date	17 July 2026
Planning horizon	2026-2035
Primary domains	Manufacturing operations and education operations
Reference scale	10,000 connected users; 1 TB authoritative relational estate; independent high-volume telemetry
Name	Actualis Stack - working name; legal clearance outstanding

Decision language

Label	Meaning
Accepted baseline	Foundation to use for the first implementation unless new evidence overturns it
Proposed	Preferred direction that still requires a benchmark, customer fact or legal decision
Deferred	Not rejected; excluded until a named quality scenario justifies its cost
Hard limit	Safety, authority or privacy boundary that implementation may not weaken

Primary readers are product leadership, domain experts, architects, security/privacy leaders, experience designers and the first engineering teams. Product and domain reality take precedence over this document; changed evidence should produce a new ADR and a revised version, not a hidden exception.

Contents

Executive decision	2
The first architecture decision	2
The implementation hypothesis	2
How to read this paper	3
Document control	3
Decision language	3
Contents	4
The initial problem	8
Operational reality in manufacturing	8
Operational reality in education	8
Reference design envelope - provisional, testable, revisable	9
What future-proof means	10
Reality-first architecture loop	10
Seven non-negotiable principles	10
Lessons from incumbent platforms	11
The shared failure patterns	11
Product thesis and boundaries	12
Why the name Actualis	12
Sharp, but composable	12
The conceptual foundation	13
A minimal operational world model	13
Commitment lifecycle	13
A governed capability is the architectural atom	13
Logical architecture	14
Core organization: modular monolith first	14
Extraction rule	14
Data architecture for 1 TB and beyond	15
The observation promotion rule	15
PostgreSQL at 1 TB	16
Machine and edge communication	17
Protocol stance	17
Actualis Edge responsibilities	17
Safety boundary	17
Stakeholder and system communication	18
Every caller and recipient is a principal	18
Actualis Relay	18

Experience architecture: UI is first-class	19
The open boundary and the opinionated path	19
Client machine as a participant	19
Generated UI and AI UI	20
AI-native without AI authority	20
Two AI modes	20
Four governed roles and five forms of intelligence	20
AI request pipeline	21
Semantics, insights and the Challenge Engine	21
Business semantic layer	21
Insight is a first-class object	21
Question Graph	22
Attention economics	22
Security, privacy and governance	23
Privacy by architecture	23
High-consequence threat cases to test first	24
Permission architecture: fine-grained and usable	25
The authorization decision	25
Layered evaluation - hard boundaries first	25
Permission authoring and operations	26
Easy for developers; safe for administrators	26
Illustrative policy intent	26
Delegation, exceptions and emergency access	26
Enforcement and performance	27
Permission invariants	27
Concurrency, reliability and server economics	28
Workload lanes	28
Cost and stress controls	28
Provisional service objectives	28
Cell architecture and deployment	29
Deployment topology	29
Release compatibility	29
Rapid environments, promotion and redundancy	30
Environment ladder	30
Fast sandbox contract	30
Build once, promote with evidence	31
Redundancy and continuity tiers	31
Continuity tiers	31
Why not global active-active relational writes by default	32
Release and recovery exercises	32

Provisional deployment objectives	32
Technology decisions - consequences of the needs	33
Technology deliberately deferred	33
Integration and protocol architecture	34
Interaction patterns, not protocol religion	34
Protocol adapter contract	35
Webhooks, JSON-RPC and future adapters	35
Outbound webhooks	35
Inbound webhooks	35
JSON-RPC 2.0	35
Future protocol rule	36
Integration sandbox	36
Contract lifecycle and deprecation	36
Domain packages and product surfaces	37
Domain source and generated infrastructure	37
Three reference journeys	38
Journey A - manufacturing exception and replan	38
Journey B - teacher absence and timetable change	38
Journey C - branded content and commerce	38
Implementation plan	39
Phase 0 - reality contracts	39
Phase 1 - the walking skeleton	39
Phase 2 - three proof slices	39
Phase 3 - hardening before expansion	39
Repository, engineering workflow and CI	40
Suggested repository shape	40
Mandatory pipeline gates	40
Team ownership	40
Architecture decision records	41
Selective temporality decision	41
Risks, countermeasures and kill criteria	42
Decisions still open	42
First 90 days	43
Definition of success at day 90	43
Appendix - quality scenarios	44
Appendix - example capability contract	45
Contract commentary	45

Appendix - evaluation scorecards	46
Architecture fitness scorecard	46
AI evaluation scorecard	46
Appendix - glossary	47
Appendix - sources and standards	48
Conclusion	50

PART 02

The initial problem

The original request appears conventional: choose a stack for high concurrency, strong cybersecurity and privacy, server efficiency, a highly relational database, IoT connections and mobile interfaces. The deeper requirement is unconventional: design a foundation that still makes sense in 2035 without freezing 2026's application paradigms.

Manufacturing and education expose why a normal ERP or LMS framing is insufficient. Their core work is not merely create-read-update-delete plus workflow. They continuously reconcile observed reality, intentions, commitments, authority, policy, constraints, objectives, decisions, execution and evidence.

Operational reality in manufacturing

- Plants must coordinate orders, material, machines, tools, skills, shifts, maintenance, quality holds, changeovers, energy and delivery commitments.
- RFID readers, scanners, PLC-adjacent systems and sensors are intermittent, heterogeneous and not inherently trustworthy. A plant may continue operating while the cloud is unavailable.
- Plans become obsolete as soon as a machine fails, a batch is held or a priority changes. Replanning must explain trade-offs and respect authorization.
- Traceability is evidentiary: who observed what, which version of a plan was dispatched, what was executed, and why an exception was accepted.
- Raw telemetry can dwarf transactional data and has different retention, query and cost characteristics.

Operational reality in education

- Timetables coordinate students, cohorts, teachers, qualifications, rooms, curricula, support sessions, contractual workloads, safeguarding and wellbeing.
- Thousands of learners, families and staff can arrive at the same time. Their views, permissions and interaction patterns differ radically.
- Student and family data is highly sensitive; purpose limitation, data minimization, retention, support access and regional isolation are architectural concerns.
- A proposed schedule is not the published schedule. Changes require authority, communication, acknowledgement and a reconstructable decision trail.
- The same school domain may need a calm student mobile experience, an information-dense planner workbench, a guardian portal and a public site.

Reference design envelope - provisional, testable, revisable

Dimension	Provisional envelope	Architectural consequence
Users	10,000 simultaneous authenticated connections; active-user burst to be measured	Push/delta sync, bounded pools, workload isolation, reconnect control
Relational data	1 TB authoritative estate over multiple cells	Partitioning, replicas/projections, archival, migration rehearsal, cell placement
Telemetry	Independent high-volume stream; rate and retention per site	Edge batching, specialized store, promotion of validated observations
Availability	Operations continue through intermittent connectivity	Local projection, durable edge queues, explicit reconciliation
Latency	Local UI feedback under 100 ms; ordinary server actions target p95 under 250 ms	Optimistic local state only where safe; server remains authority
Privacy	Minor, employee, customer and device data with different purposes	Classification, minimization, retention, cell isolation, policy tests
Change	Multiple UIs per domain; technology replacement over ten years	Headless contracts, experience packages, versioned schemas and events

IMPORTANT

Ten thousand users is not a useful performance specification by itself. Before capacity planning, measure connected sessions, active interactions per second, websocket fan-out, command rate, sync payload, solver jobs, AI tasks and telemetry events separately.

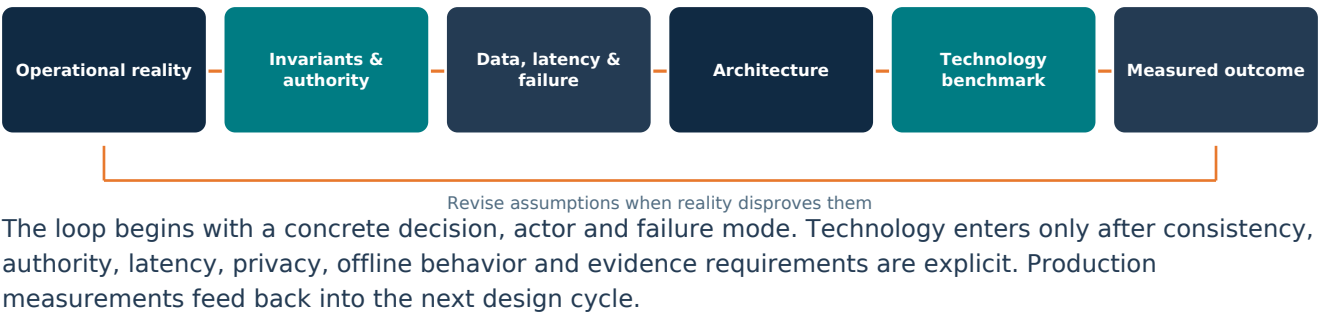
PART 03

What future-proof means

Future-proof does not mean predicting the winning framework in 2035. It means localizing change, preserving evidence, and ensuring that domain meaning and authority outlive infrastructure choices.

Not future-proofing	Actual future-proofing
A universal entity model	Stable domain contracts and explicit migration paths
Supporting every database	Assigning each data class to an owner with a narrow integration contract
A proprietary UI renderer	Independent experience packages over open HTTP, realtime and sync contracts
A permanent AI provider abstraction	Task contracts, evaluations, context packages and reproducible tool traces
Microservices everywhere	Modular ownership and extraction only where failure or scale is truly independent
Metadata for everything	Typed source, generated boring infrastructure and handcrafted differentiating behavior
Never changing schemas	Versioned schemas, compatibility windows and rehearsed migration

Reality-first architecture loop



Seven non-negotiable principles

- Domain meaning is specialized. Share behavioral contracts and engines; do not pretend a teacher and a machine are the same entity.
- Consequential actions pass through governed capabilities with explicit authority, invariants, effects, evidence and risk.
- Authoritative truth is relational and domain-owned. Projections may be disposable; evidence is durable.
- User experience is a first-class product artifact, not a generated form around a schema.
- The edge may observe, buffer and execute authorized commands, but the cloud platform is not a safety controller.
- AI can interpret, analyze, challenge, explain and propose. It cannot grant itself authority, rewrite facts or approve its own action.
- The smallest architecture that passes a measured operational scenario wins.

PART 04

Lessons from incumbent platforms

Actualis should learn from the strengths of incumbents without copying the coupling that creates their long-term pressure. The observations below describe recurring architectural trade-offs, not blanket judgments; deployments and product editions vary.

Platform	What to retain	Pressure to avoid	Actualis response
SAP	Process depth, controls, master-data rigor, enterprise accountability	Custom-code and integration debt can slow upgrades and raise operating cost; SAP itself promotes clean-core separation [S1][S2]	Stable capability boundary, cell-owned core, separately deployed extensions, evidence by default
Odoo	Broad module velocity, productive ORM, pragmatic business coverage	Model inheritance, module dependencies and upgrade scripts make customization and migration a continuing concern [S3][S4][S5]	Typed contracts, explicit migrations, no monkey-patching across module ownership
Frappe	Fast scaffolding, approachable metadata, automatic APIs and admin UI	DocType is model and view center; generic CRUD can become the interaction ceiling [S6][S7][S8]	Capabilities at the center; generated admin is fallback; independent experiences are primary
Framework M	Code-first typing, ports/adapters, multitenancy and modular-monolith intent	Still DocType-centered and very young; broad adapter neutrality can become lowest-common-denominator architecture [S9][S10][S11]	Copy seams and ergonomics; keep domain decisions, commitments and evidence as the center
PowerSchool	Comprehensive SIS, institutional workflows, ecosystem reach	Concentrated sensitive data and privileged access pathways create a high-consequence target; its 2024 incident is a warning [S12]	Cells, minimization, JIT support access, fine-grained authorization, per-cell evidence and keys
ManageBac	Coherent IB domain experience and useful integration APIs	A canonical workflow and surface cannot serve every stakeholder or school identity; integrations require governance [S13][S14]	Deep education semantics with multiple independent surfaces and governed integration principals
Canvas	Open REST API, OAuth and interoperability ecosystem	A course/content container should not become the universal operational model; integration sprawl can fragment authority [S15]	Keep learning interoperability at the boundary; operational commitments remain in owned domain modules
Shopify pattern	Headless commerce, strong checkout boundary, theme and app ecosystem	Commerce concepts should not dictate manufacturing or education core semantics	Commerce is a domain/experience pack over capabilities, not the platform center

The shared failure patterns

Coupling pressure

- One data model also drives API, UI and permissions
- Customizations invade the upgrade path
- A generic workflow hides policy, authority and evidence
- Every workload competes for the same database and workers

Operational pressure

- One canonical screen becomes the product
- Integrations bypass domain invariants
- Support access is too powerful
- AI is chat over raw data instead of a governed actor

LESSON

Do not build an Odoo replacement by adding a better solver to DocTypes. Build the authority, decision, execution and experience boundaries that CRUD-centered platforms do not make primary.

PART 05

Product thesis and boundaries

Actualis helps constraint-intensive organizations continuously observe, plan, commit, execute, explain and adapt their operations.

Why the name Actualis

Actualis points to actual operational reality and to action. It avoids tying the platform to ERP, manufacturing, school administration or a particular technology. The tagline Systems shaped by reality states the design doctrine. The name is provisional; formal trademark, corporate-name and domain clearance is required.

Working component	Responsibility
Actualis Core	Authority, capabilities, commitments, policies, operational state, execution and evidence
Actualis Surface	Experience contracts, SDK, local projections, accessibility and surface deployment
Actualis Edge	Machine protocols, device identity, buffering, offline execution and reconciliation
Actualis Signal	Telemetry ingestion, data quality, validated observations and analytical projections
Actualis Decide	Scenario formulation, solver portfolio, simulation, comparison and infeasibility explanation
Actualis Guide	Semantic layer, insight store, question graph, AI context and bounded AI tools
Actualis Relay	Stakeholder audiences, channels, acknowledgement, escalation and external delivery
Actualis Link	Public APIs, webhooks, JSON-RPC, protocol adapters, contract catalog and integration sandboxes

Sharp, but composable

Actualis is excellent at

- Accurately modeling operational commitments and authority
- Finding feasible plans and explaining infeasibility
- Replanning when observations invalidate assumptions
- Executing decisions across people, systems and devices
- Reconstructing why consequential decisions occurred
- Presenting one domain through radically different experiences

Actualis does not claim to be

- A universal ontology or generic low-code platform
- A replacement for PLC, SIS safety logic or real-time machine control
- The best accounting, payroll or general office suite
- A universal CMS or commerce engine at its core
- A database portability layer
- Autonomous AI operating without accountable humans

MES, APS, task scheduling, education operations, sites and commerce can all be built on Actualis because they consume capabilities and domain packages. That breadth is a consequence of clean boundaries, not a promise that every problem is reduced to the same primitive.

PART 06

The conceptual foundation

A minimal operational world model

Group	Semantic interfaces	Question answered
World	Actor, entity, observation, capability, commitment	What exists, what is believed, what can be done, and what has been promised?
Governance	Authority, policy, constraint, objective	Who may decide, what is mandatory, and what should be optimized?
Decision	Intent, scenario, plan, decision, evidence	What outcome is sought, which future was considered, and why was one alternative selected?
Execution	Command, event, process	What was requested, what happened, and what durable coordination remains?

These are semantic interfaces, not a command to create one generic table for every entity or observation. Stable, performance-sensitive domains use strongly relational schemas. Shared engines consume narrow typed views.

Commitment lifecycle

Every transition is an authorized capability with explicit evidence and consequences.



A proposal is not a promise. A generated schedule is not a published commitment.

A governed capability is the architectural atom

A capability is not an endpoint name. It is a typed contract that combines action, authority, operational invariants and consequences. Every human, device, integration, background worker and AI agent invokes the same capability boundary as an identified principal.

Facet	Required definition
Intent	Typed command or query and its business meaning
Authority	Principal, role/relationship, purpose, scope and approval requirement
Invariants	Domain conditions that must remain true
Effects	Authoritative changes, emitted events, projections and notifications
Evidence	Input provenance, decision explanation, actor, time, versions and outcome
Risk	Impact class, reversibility, blast radius, rate limit and step-up controls
Delivery	Idempotency, expected version, retry behavior, offline allowance and conflict rule
AI policy	Whether AI may read, draft, propose, execute or never access the capability

PART 07

Logical architecture

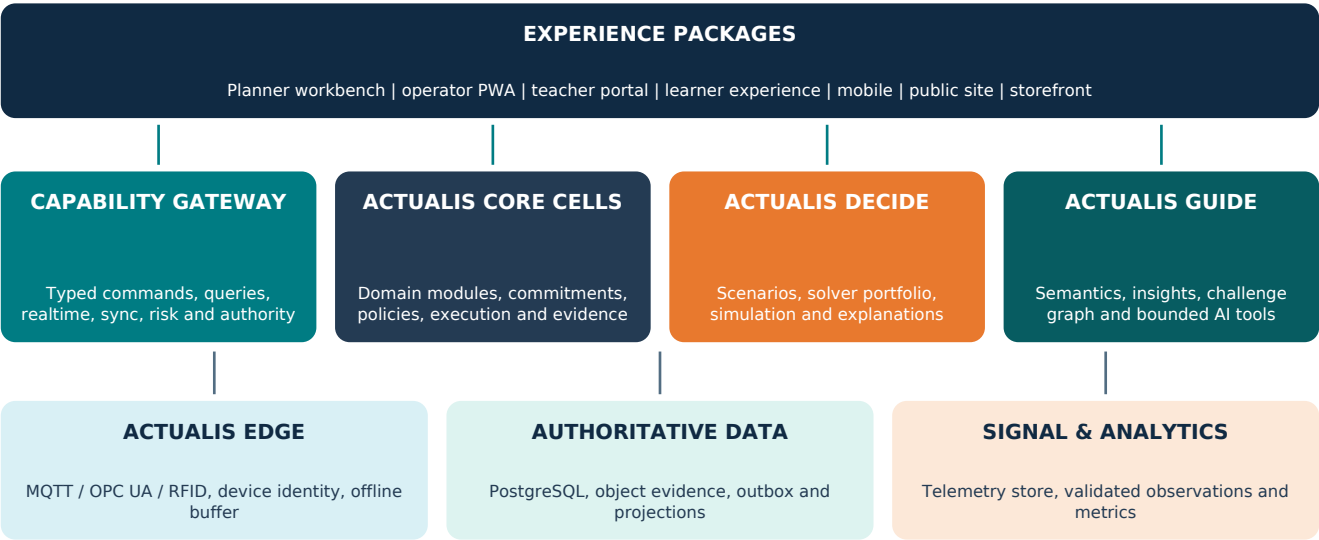


Figure 1. Actualis logical architecture. Lines represent governed contracts, not shared database access.

Core organization: modular monolith first

The first production core is a modular monolith deployed as cells. Each module owns its schema, invariants, policies and capabilities. Cross-module calls use in-process typed contracts; externally visible changes use versioned events. This keeps important transactions local and debugging tractable.

Core module	Owns	Does not own
Identity & Authority	Principal links, relationships, purposes, grants, policy evaluation	Identity-provider credentials or UI routing
Commitments & Calendars	Reservations, obligations, effective dates, availability	Factory-specific routing or curriculum semantics
Decision & Scenario	Intent, snapshot, candidate plans, selection, explanation	Solver implementation
Execution	Commands, dispatch, acknowledgement, compensation, durable jobs	Machine safety control
Evidence	Audit chain, provenance links, attachments, retention class	Raw telemetry payloads
Communication	Audience, preferences, channel request, acknowledgement, escalation	Email/SMS provider internals
Domain packs	Manufacturing, education and commerce semantics and invariants	Generic kernel behavior

Extraction rule

A module becomes a separate deployable only when at least one condition is demonstrated: independently extreme resource profile, different safety/security boundary, independently required availability, external network placement, or deployment cadence that materially conflicts with the core. Edge, telemetry, solver workers and AI workers meet this test earlier than ordinary business modules.

ANTI-PATTERN

Do not turn every module boundary into a network call. A clean code seam is valuable even when it never becomes a microservice.

PART 08

Data architecture for 1 TB and beyond

Actualis does not pursue database neutrality. It assigns each data class to a clear owner based on consistency, access pattern, retention and failure semantics. The integration contract is stable; the physical store is deliberately chosen.

Data class	Authoritative owner	Consistency and use
Transactional truth	PostgreSQL per cell	Strong relational constraints; commands, commitments, approvals, current state
Validated observation	PostgreSQL domain schema	A promoted operational fact with source, quality, time and evidence reference
Raw telemetry	Actualis Signal store; ClickHouse default candidate	High-volume append, downsampling, time-window analytics; not direct transactional authority
Evidence/media	Versioned object storage	Immutable or retention-managed documents, images, payloads and signed artifacts
Analytical projection	Read replica / ClickHouse projection	Rebuildable facts and governed metrics for dashboards and AI analysis
Search index	PostgreSQL FTS first; external index when proven	Disposable discovery projection, never authority
Client projection	SQLite/OPFS or native SQLite	Surface-specific, scope-limited, replaceable and syncable
AI artifacts	PostgreSQL metadata + object evidence	Prompt/context hashes, model/tool traces, insight objects and evaluation outcomes

The observation promotion rule

No consequential decision may depend on an uncaptured volatile query to a remote or specialized store.

A planner may explore raw telemetry, but a quality hold, schedule commitment or compliance decision must reference a validated observation captured in the authoritative domain. The observation records source, event time, ingest time, quality status, transformation, evidence pointer and version.

Stage	Responsibility	Failure treatment
Ingest	Authenticate device/gateway; assign idempotency and timestamps	Quarantine unknown device, retain raw payload
Normalize	Decode protocol payload into typed measurement/event	Record schema/version error without inventing values
Validate	Check calibration, range, sequence, clock quality and business context	Mark uncertain or rejected; do not silently promote
Promote	Create domain observation with provenance and evidence link	Transactional retry via inbox/outbox

Stage	Responsibility	Failure treatment
React	Evaluate policies, constraints and affected commitments	Create proposed action; high-impact changes require approval

PostgreSQL at 1 TB

One terabyte is not itself a reason to abandon PostgreSQL. The design uses multiple cells, bounded connection pools, declarative partitioning where access patterns justify it, read replicas or projections for analytical reads, and explicit archival of cold evidence and history. Row-level security can provide defense in depth, but domain authorization remains in capability policy [S17][S18][S19].

- Partition by stable operational boundaries such as cell and time, not by every tenant reflexively.
- Keep hot indexes small; avoid indexing every flexible attribute; measure query plans continuously.
- Rehearse online migrations against production-shaped data and keep expand/migrate/contract compatibility windows.
- Route reporting and AI analytics to governed projections, not the primary command path.
- Never attempt distributed transactions across stores. Use outbox/inbox, idempotency, reconciliation and compensating capabilities.

PART 09

Machine and edge communication

The edge is a trust and continuity boundary. It speaks plant protocols, survives unreliable connectivity, minimizes cloud traffic and converts device traffic into typed observations. It does not turn the enterprise platform into a deterministic safety controller.

Path	Flow	Rules
Observation	Device -> gateway -> raw stream -> validation -> promoted observation	At-least-once tolerated through idempotency; preserve source and event time; never assume arrival order
Command	Authorized capability -> dispatch -> gateway -> adapter -> acknowledgement	Explicit expiry, expected state, risk class and allowed offline execution; no silent retry of unsafe commands
Configuration	Signed desired config -> gateway validation -> staged activation	Versioned, reversible where possible, device compatibility checked
Evidence	Gateway log/payload -> object storage -> evidence reference	Tamper-evident hash, retention class and clock-quality metadata

Protocol stance

Use OPC UA where machine semantics and industrial interoperability require it; MQTT 5 for lightweight publish/subscribe transport; vendor SDKs and serial/USB adapters only behind gateway drivers; RFID and barcode readers as authenticated device principals. MQTT and OPC UA are boundary protocols, not domain models [S20][S21].

Actualis Edge responsibilities

Observation continuity

- Per-device identity and certificate lifecycle
- Protocol decoding and schema versioning
- Sequence, deduplication and clock-quality tagging
- Durable local buffer with quotas and backpressure
- Batching, compression and bandwidth policy

Governed execution

- Local read models for allowed offline work
- Signed command verification and expiration
- Health, inventory and software attestation
- Safe reconciliation after reconnect
- Remote diagnostics with tightly scoped support grants

Safety boundary

HARD LIMIT

Actualis may schedule, dispatch and observe industrial work. Certified PLC/SIS logic retains real-time interlocks and safety authority. If connectivity or Actualis fails, machinery must transition according to its certified local controls.

PART 10

Stakeholder and system communication

Communication is part of execution, not a side effect. A timetable change that reaches only half the students, or a quality hold that does not reach the operator, is an operational failure even if the database row is correct.

Every caller and recipient is a principal

Principal type	Examples	Control
Human	Planner, operator, teacher, learner, guardian, administrator, support engineer	Identity, relationship, role, purpose, device and step-up state
Device	RFID reader, edge gateway, scanner, sensor adapter	Certificate, facility scope, protocol, firmware posture and rate
Integration	ERP, HR, LMS, supplier, payment or identity system	Client identity, capabilities, data purpose, schema/version and quota
Background worker	Scheduler, exporter, notification worker	Workload identity, narrow capability and trace context
AI agent	Analyst, adviser, challenger, companion	Task contract, context boundary, tool grants, risk policy and full trace

Actualis Relay

Stage	Required behavior
Audience resolution	Resolve roles and relationships at send time; record the audience rule and resulting recipients
Policy	Apply privacy, consent, purpose, quiet hours, urgency, localization and accessibility preferences
Composition	Use versioned templates and domain evidence; arbitrary free text cannot become a high-impact instruction
Channel selection	In-app, push, email, SMS, voice, webhook or display by urgency and recipient capability
Delivery	Track provider request, receipt, bounce, retry, deduplication and cost
Acknowledgement	Distinguish delivered, seen, understood and accepted; escalate when the commitment requires it
Audit	Preserve who authorized communication, which facts it contained and who received each version

Domain modules request a communication outcome; they do not call an email or SMS provider directly. Relay adapters may change without changing the domain event. The same mechanism supports school closure notices, maintenance escalations, planner approvals, supplier messages and device acknowledgements.

PART 11

Experience architecture: UI is first-class

A domain is not its screen, and a screen is not a generated view of a table.

Actualis treats an experience package as an independently versioned product surface with its own navigation, interaction model, local projection, performance budget, release cadence and accessibility evidence. It consumes governed capabilities; it does not own domain invariants.

Package	Examples	Can differ
Planner workbench	Dense timeline, conflict graph, scenario comparison, keyboard workflows	Information architecture, visualization, interaction and approval flow
Operator PWA	Scan-first tasks, large controls, glove mode, offline queue, station context	Device integration, offline scope, language and visual hierarchy
Teacher / learner / guardian	Calm mobile schedule, attendance, notices, tasks, accessibility	Entire experience and branding while sharing domain capabilities
Public site	Static/SSR content, course or product discovery, forms	Framework, rendering strategy, content model and design system
Storefront	Catalog, configuration, reservation, checkout handoff	Brand, merchandising and commerce interaction; commitments remain governed

The open boundary and the opinionated path

Fast and safe by default

- TypeScript contracts and generated clients
- React-based component and interaction SDK
- Realtime/sync client and local projection toolkit
- Authorization-aware action components
- Accessible primitives, testing harness and observability
- Optional design tokens and reference themes

No ceiling on developer imagination

- Documented HTTP, websocket and event contracts
- Any UI framework or native client may be used
- Trusted developers may author arbitrary styles and interactions
- Signed artifacts and contract conformance remain required
- No proprietary renderer is required to access the domain

Client machine as a participant

The client stores only its authorized surface projection, not a copy of the enterprise database. Web clients use Service Workers for offline assets and sync, SQLite compiled to WebAssembly with OPFS for durable local state, and Web Workers/WASM for expensive filtering, layout, validation, diffing or small simulations [S22][S23][S24]. Native clients use the same projection/sync contract.

Mobile path

Use the installable PWA first when it can satisfy offline, camera/scanning, notification and device management needs. Use Flutter for the official native operational shell when reliable background work, NFC/BLE, kiosk controls or deeper platform integration justifies it. Swift, Kotlin or another client remains possible because capability and sync contracts - not Flutter - define compatibility.

Client may	Client may not
Render immediately from a local projection	Declare a consequential action committed without server or delegated edge authority
Queue explicitly offline-allowed commands	Cache data outside its purpose, scope or retention window
Perform search, sort, validation and scenario previews	Override policy, permissions, hard constraints or authoritative versions
Apply optimistic UI for reversible low-risk actions	Hide conflicts or silently resolve concurrent edits
Use WASM for deterministic portable compute	Block the interaction thread with solver, AI or large transforms

Generated UI and AI UI

Generated forms are useful for administration, diagnostics and early scaffolding. They are not the primary product experience. Build-time AI may generate reviewed source and tests. Runtime AI may compose only registered, permission-aware components from typed results; it cannot inject arbitrary JavaScript into a privileged surface. Content Security Policy and signed package controls protect the shell [S30].

PART 12

AI-native without AI authority

AI is designed into the continuous decision loop, but the deterministic system remains complete without it. If every model is unavailable, users can still scan, schedule, enforce constraints, approve, dispatch, execute and audit.

Two AI modes

Mode	Purpose	Boundary
Build-time AI	Draft domain source, migrations, tests, UI, documentation, threat cases and evaluation fixtures	Normal code review, CI, signing and deployment; no direct production mutation
Runtime AI	Interpret intent, analyze governed metrics, challenge assumptions, explain solver results, draft scenarios and propose capabilities	Identified principal, typed tools, scoped context, risk policy, validation and evidence

Four governed roles and five forms of intelligence

Role	Contribution	Must not do
Analyst	Describe and diagnose changes through governed metrics and approved analytical views	Invent metric definitions or run unrestricted production SQL
Decision adviser	Reframe intent, expose stakeholders and formulate a typed solver problem	Pretend its narrative is a feasible plan
Constructive challenger	Ask what assumption, cohort, policy, cost or alternative explanation is missing	Manufacture controversy or state correlation as cause
Operational companion	Explain context and next allowed actions inside the user's workflow	Reveal information outside the user's purpose and relationships

Across these roles, Actualis supports descriptive, diagnostic, predictive and prescriptive intelligence, plus reflective intelligence: helping the organization discover the important question it has not yet asked.

AI request pipeline

Stage	Contract
1. Context	User, role, purpose, cell, intent, world snapshot, policies, evidence and allowed tools
2. Interpretation	Model produces a typed analytical or decision intention, never an executable side effect
3. Validation	Schema, policy, data classification, budget and risk gates run deterministically
4. Tooling	Read governed metric, create scenario, request solver, compare plans, find evidence or propose command
5. Evidence	Record model/configuration, agent/prompt version, context hashes, retrieval, calls, results and uncertainty
6. Authority	Human or deterministic policy approves any consequential capability; AI never self-approves
7. Outcome	Track acceptance, correction, execution and eventual operational impact for evaluation

Internal typed commands and queries are canonical. MCP may be exposed as an external adapter, but it does not define the domain architecture or bypass the same authorization policy [S33].

PART 13

Semantics, insights and the Challenge Engine

Business semantic layer

AI and dashboards must share governed definitions. A semantic metric records definition, grain, units, time basis, filters, exclusions, data owner, security class, quality state, version and calculation provenance. The AI may choose a governed metric; it may not secretly redefine it.

```
metric: schedule_adherence
definition: completed_on_time / planned_operations
grain: production_operation
time_basis: actual_completion_time
exclusions: [cancelled, engineering_trial]
owner: production_planning
quality_status: certified
version: 3
```

Insight is a first-class object

An insight is not a transient chat bubble. It is a versioned claim linked to evidence, confidence, alternative explanations, affected commitments, generating agent and required reviewer. Users can accept, reject, correct, investigate or convert it into a scenario. Outcome tracking makes future evaluation possible.

Field	Example
Claim	Capacity shortage risk on mould family X within 14 days
Evidence	Committed load 112%; maintenance window M231; demand forecast v18
Uncertainty	Medium confidence; forecast and maintenance duration vary
Alternatives	Outsource operation; move maintenance; renegotiate delivery
Affected commitments	Customer orders 847 and 851
Provenance	Agent version, model configuration, prompt, context and tool trace
Review	Production planner; expires when source snapshot changes materially

Question Graph

The Challenge Engine traverses curated question patterns instead of improvising provocation: What changed? Compared with what? For whom? Which commitment is affected? Which constraint is binding? Who benefits and who carries the cost? What data is missing? What happens if we do nothing? Is the action reversible? Domain packs add manufacturing and education questions.

Manufacturing questions

- Is utilization hiding excess work in progress?
- Did lower defects coincide with lower inspection coverage?
- Are expedited orders degrading the total schedule?
- Did a local efficiency gain move the bottleneck downstream?

Education questions

- Does an average hide a struggling cohort?
- Did timetable improvement shift workload unfairly?
- Are absences concentrated after schedule changes?
- Does feasibility conflict with wellbeing or safeguarding?

Attention economics

Rank insights by importance x confidence x actionability x time sensitivity x novelty x scope, divided by interruption cost. Suppress duplicates, known seasonal effects and accepted recurring risk. Choose among immediate interruption, daily briefing, weekly review, silent evidence gathering or a question at the next decision point.

PART 14

Security, privacy and governance

Actualis follows zero-trust principles: network location is not trust, access is evaluated per resource and action, and every workload has an identity [S25]. Authentication uses an external OpenID Connect provider; authorization remains a domain responsibility [S26].

Layer	Baseline controls
Identity	OIDC, phishing-resistant MFA/passkeys where feasible, workload identities, device certificates, session risk and step-up
Authorization	Capability-level relationship and attribute policy; purpose and cell scope; deny by default; RLS as defense in depth
Tenancy	Cells by tenant/regulatory/operational boundary; separate keys, queues and storage prefixes; minimal business data in control plane
Data protection	Classification, minimization, purpose limitation, retention, legal hold, export/deletion and field-level redaction
Support	Just-in-time scoped grant, customer approval for sensitive access, session recording, dual audit and break-glass review
Edge	Per-device identity, signed software/configuration, local secret protection, quarantine, inventory and revocation
AI	Context filtering, tool allowlists, prompt-injection isolation, output validation, no direct SQL, DLP and reproducible traces
Supply chain	Pinned dependencies, SBOM, signed builds/images/packages, provenance, secret scanning and rapid revocation
Operations	Immutable audit/evidence, anomaly detection, incident runbooks, cell isolation, backup restore and recovery exercises

Privacy by architecture

For education, the system must assume that a minor's data can be harmful when over-collected, over-retained or shown to the wrong relationship. For manufacturing, employee, supplier and customer data also has purpose boundaries. Data protection principles such as lawfulness, purpose limitation, minimization, accuracy, storage limitation, integrity and accountability inform the default design [S27][S28].

- Every field receives a classification, purpose, owner and retention class in domain source.
- Client projections include only data needed for the current surface and are expired or revoked explicitly.
- Analytics and AI receive de-identified or aggregated views where identity is unnecessary.
- Evidence retention can differ from operational record retention; deletion preserves required audit without excess content.
- Cross-cell operations are explicit export/import capabilities with evidence, not shared superuser queries.

High-consequence threat cases to test first

Threat	Mandatory test
Cross-tenant leak	Every capability, query, sync delta and AI context respects cell, relationship and purpose scope
Support compromise	A stolen support account cannot access customer data without JIT grant, approval and recorded session
Device spoofing/replay	Unknown or revoked certificate and duplicate sequence cannot create a trusted observation
Prompt injection	Untrusted document text cannot expand AI tool authority or exfiltrate hidden context
Offline theft	A lost device exposes minimum projection; revocation prevents new sync and local expiry reduces residual data
Solver/AI saturation	User commands retain SLO through separate queues, pools and admission control

PART 14A

Permission architecture: fine-grained and usable

Permissions are part of the domain contract. They must be precise enough for real operations and simple enough that authorized administrators can understand, test and change them safely.

Traditional role-based access control is necessary but insufficient. A person may be a teacher, planner or operator and still be authorized only for particular students, sites, machines, shifts, records, purposes or fields. Actualis therefore combines role-, relationship-, attribute-, purpose-, context- and risk-aware authorization around each governed capability.

The authorization decision

Input	Examples	Question
Principal	Human, device, integration, workload or AI; verified identity and assurance	Who or what is asking?
Capability	View support plan, move material, approve schedule, export records	What exact action is requested?
Resource relationship	Assigned teacher, guardian of learner, operator at site, support grant	What legitimate relationship connects the principal to the subject?
Attributes	Qualification, department, data class, quality state, age/minor status	Which facts narrow or prohibit access?
Purpose	Teach assigned class, execute production, investigate incident, provide support	Why is the information or action needed?
Context	Cell, site, device posture, location, shift, time, network and session risk	Under what current conditions is access safe?
Decision risk	Impact, reversibility, volume, sensitivity and separation of duties	Is approval, step-up, masking or a lower limit required?
Policy version	Effective-dated domain and customer policy set	Which rules must be reproduced in the evidence trail?

Layered evaluation - hard boundaries first

Order	Layer	Result
1	Cell and tenant boundary	Reject any cross-cell access unless an explicit exchange capability authorizes it
2	Capability grant	Determine whether this principal type may invoke this business action at all
3	Relationship and scope	Limit records to assigned students, sites, teams, machines, orders or cases
4	Purpose and consent	Require an allowed use; enforce consent or legal basis where applicable
5	Field and projection policy	Remove, mask, aggregate or watermark sensitive fields before delivery
6	Context	Evaluate time, device trust, location, shift, assurance and current risk signals
7	Separation and approval	Require another authorized principal for high-impact or conflicting duties
8	Obligations	Log evidence, notify an owner, impose expiry, rate limit or require acknowledgement

The result is not only allow or deny. It may be allow with obligations: redact a field, limit the row set, require step-up authentication, request approval, watermark an export, impose a maximum volume, record a reason or expire the grant. Denial always returns a safe machine-readable reason and a human explanation without leaking protected facts.

PART 14B

Permission authoring and operations

Easy for developers; safe for administrators

Surface	Framework capability
Typed policy manifest	Domain package declares capabilities, resource types, relationships, attributes, purposes, fields, risk and default obligations
Policy templates	Manufacturing, education, support, guardian, device and integration starting policies with safe defaults
Visual Policy Studio	Decision tables and guided rules using domain language; no need to edit application code for ordinary scope changes
Access simulator	Choose a principal, capability, resource and context; see allow/deny, redaction, obligations and the exact rule path
Effective-access explorer	Answer who can do this, what can this person do, why, since when and until when
Policy diff and approval	Human-readable change impact, affected principals/resources, peer approval and effective date before activation
Generated tests	Positive, negative and boundary cases for every declared relationship, purpose, field and offline rule
Runtime explanation	Stable reason codes and safe narratives for UI, support, audit and AI tool decisions

Illustrative policy intent

```
policy: education.support_plan.read
allow_when:
- principal.is_assigned_teacher_of(student)
- purpose == deliver_current_instruction
- relationship.is_current_at(request_time)
fields:
general_adjustments: allow
safeguarding_notes: deny_unless(safeguarding_role)
obligations:
- record_access_reason
- expire_client_projection_after: 8_hours

policy: manufacturing.material.issue
allow_when:
- principal.is_assigned_to(site)
- principal.has_current_qualification(material_class)
- device.is_managed_station_at(site)
- current_shift.includes(principal)
deny_when:
- batch.quality_status == hold
offline:
allowed: false
```

Delegation, exceptions and emergency access

- Delegation is typed, scoped, time-limited and accepted by the delegate; it never copies an entire role silently.
- Substitute teachers, temporary operators and contractors receive task- or relationship-scoped access that expires automatically.

- Break-glass access requires a reason, stronger authentication, narrow duration, prominent evidence and mandatory after-action review.
- Support access is a customer-approved just-in-time grant to named capabilities and data classes, never a permanent global superuser.
- Revocation invalidates future server and sync decisions immediately; offline projections and queued commands carry explicit expiry and are re-authorized on commit.

Enforcement and performance

Actualis Core owns the canonical authorization model and evaluates it before every capability. Domain queries compile the decision scope into bounded relational predicates; projection builders apply field policy before data leaves the cell. PostgreSQL row security is an additional containment layer, not the only policy engine. Realtime subscriptions, exports, Relay audiences, edge commands and AI tools all use the same decision contract.

The relationship graph and policy versions begin in PostgreSQL inside each cell. Safe, short-lived decision caches may be used only when revocation, policy version and context are part of the cache key. High-risk actions, sensitive fields and support/break-glass decisions are never authorized from a stale cache.

FRAMEWORK PROMISE

A developer declares the security vocabulary once. Actualis generates typed hooks, policy tests, administration screens, simulator inputs, explanations, audit evidence and client redaction contracts. Domain owners still decide the policy; the framework makes the secure path the easiest path.

Permission invariants

- Default deny: a new capability, field, principal type or relationship is inaccessible until its policy is explicit.
- No UI-only security: hiding a button never substitutes for server-side capability evaluation and projection filtering.
- No bypass path: imports, reports, background jobs, devices, support tools and AI use the same authorization decision contract.
- No self-grant or self-approval: policy administration and high-impact execution preserve separation of duties.
- Every allow, deny, redaction, delegation and break-glass decision records the effective policy version and safe reason code.

PART 15

Concurrency, reliability and server economics

The architecture optimizes cost by avoiding unnecessary work, not by sacrificing correctness. Client compute, static delivery, delta sync, bounded pools and workload isolation reduce server demand while making the experience faster.

Workload lanes

Lane	Examples	Isolation
Interactive command	Approve, scan, reserve, publish, acknowledge	Highest priority; short transactions; strict queue and connection budget
Realtime/sync	Presence, deltas, subscriptions, reconnect	Per-principal/channel quotas; snapshot+delta; backpressure and jitter
Integration/jobs	Imports, exports, Relay delivery, projection updates	Durable queue; idempotent workers; concurrency caps
IoT	Device ingest, normalization, observation promotion	Edge and ingest pools; raw path independent of command database
Solver	APS/timetable runs, simulation	Dedicated CPU/memory workers, time budgets, cancellation and cached scenarios
AI	Analysis, challenge, explanation	Token/cost budgets, isolated tools and queues; graceful unavailability
Reports	Large exports, analytics refresh	Replica/projection reads; scheduled and throttled

Cost and stress controls

Reduce work

- CDN/static or edge-cached delivery for public sites
- Local SQLite projection and client-side search/filter/diff
- Push and delta sync instead of polling or full refresh
- Batch/compress telemetry at the edge
- Cache immutable reference data by version
- Precompute surface projections, not massive generic payloads

Contain work

- Bound every queue, database pool and fan-out path
- Use load shedding and priority before autoscaling blindly
- Cancel superseded solver and AI jobs
- Downsample/expire raw telemetry by explicit policy
- Scale cells independently; avoid one global hot tenant
- Track cost per command, active user, device and insight

Provisional service objectives

Journey	Initial target	Exclusion / note
Local UI response	p95 below 100 ms	Render/search/filter against local projection
Ordinary online command	p95 below 250 ms, p99 below 750 ms	Excludes external provider, solver and AI completion
Realtime delta	p95 below 1 s within region	After authoritative commit; fan-out measured per surface
Offline scan acknowledgement	Immediate local receipt; sync status visible	Commit time depends on connectivity and capability policy
Interactive availability	99.95% per production cell target	Final tier by customer and operational criticality
Recovery	Authoritative core aims RPO <= 5 min, RTO <= 60 min	Tier by data class; validate through restore exercises

LOAD PRINCIPLE

Test with realistic relationship policies, websocket fan-out, sync deltas, 1 TB query plans, edge reconnect storms and migration activity. A synthetic health endpoint proves almost nothing.

PART 16

Cell architecture and deployment

A cell is the unit of business-data placement, failure containment, scaling and recovery. It contains Actualis Core application nodes, PostgreSQL authority, outbox workers, local caches and scoped projections for a bounded tenant or operational group. A light control plane manages placement, versions and health without becoming a universal business-data super-database.

Concern	Cell decision
Placement	Select by region, regulation, tenant size, plant/school group and resilience tier
Isolation	Separate database, keys, queues, object prefix and workload identity; dedicated cells for high-risk or high-volume customers
Scaling	Add application nodes within a cell; split through explicit migration rather than transparent cross-shard joins
Cross-cell	Exchange versioned events or explicit export/import capabilities; never hidden distributed transactions
Control plane	Tenant directory, routing, desired version, health and deployment policy; no broad domain-record access
Recovery	Cell-specific backup, point-in-time recovery, object replication and tested failover/runbooks

Deployment topology

Use containers and infrastructure as code, but keep Kubernetes optional until fleet size and scheduling requirements justify its cognitive cost. A cell can begin on a managed container platform with managed PostgreSQL and object storage. Edge agents use signed, staged updates with site-controlled windows.

Release compatibility

- Capability and event schemas use semantic compatibility rules and generated conformance tests.
- Database changes follow expand -> dual-read/write where necessary -> migrate -> contract.
- Experience packages declare compatible capability versions and can roll independently.
- Edge versions declare protocol/configuration compatibility and support delayed site updates.
- AI task contracts and evaluations gate model, prompt, retrieval and tool-policy changes.
- Cells support canary rollout, rollback and version-skew windows; evidence records effective versions.

PART 16A

Rapid environments, promotion and redundancy

A release should be promoted, not rebuilt. A sandbox should be disposable, not special. Redundancy should be proven through failover, not asserted in a diagram.

Actualis treats environments and recovery as products of the platform. Every cell is created from the same versioned infrastructure and application template. Differences are explicit configuration, secrets, scale and data - not hand-maintained server history.

Environment ladder

Environment	Purpose	Data and lifetime	Promotion gate
Developer workspace	Fast unit/domain work and UI development	Synthetic fixtures; local or remote dependencies; disposable	Formatting, unit, policy and contract checks
Ephemeral sandbox / preview	One branch, feature, customer demo or policy experiment	Synthetic or approved masked seed; automatic expiry	Build provenance, smoke, migration and security baseline
Integration pre-test	External API, webhook, device and provider conformance	Test identities/keys; simulators; recorded safe fixtures	Contract, replay, timeout, quota and failure-path tests
Shared test / QA	Cross-module journeys, exploratory QA, accessibility and load preparation	Curated non-production datasets; stable while a release is tested	Journey, permission, sync and regression suites
Pre-production	Production topology, migration rehearsal, failover and performance	Production-shaped masked/synthetic data; no casual access	SLO, restore, failover, penetration and release approval
Production	Real operations	Classified authoritative data; controlled retention	Canary health, error budget and accountable release decision

Fast sandbox contract

- One command or approved control-plane action creates a routable cell from a signed template, with isolated database, keys, object prefix, queues and identities.
- A normal code-only sandbox should be usable within minutes; large production-shaped datasets are a separate, asynchronous preparation step.
- Each sandbox receives OIDC test principals, device and webhook simulators, seeded domain journeys, observability and a visible expiry owner.
- Production data is never copied by convenience. Approved masking preserves relationships and distributions while removing identities and secrets.
- Cost and quota policies suspend or destroy abandoned sandboxes; evidence retains artifact versions and test outcomes without retaining excess data.

Build once, promote with evidence

Step	Required evidence
Build	Reproducible source revision, dependency lock, SBOM, signed image/package and generated contracts
Verify	Unit/domain/policy tests, static analysis, secret scan, migration plan and artifact vulnerability result
Deploy sandbox	Template version, configuration schema, seed provenance and smoke result
Promote	The same immutable digest moves to test and pre-production; only approved environment configuration changes
Rehearse	Database expand/migrate/contract, backup restore, integration replay, load and failover against production shape
Release	Canary or blue/green traffic, automated health gates, accountable approval and observable feature flags
Recover	Prefer roll-forward for schema/data changes; rollback only when compatibility and evidence prove it safe

PART 16B

Redundancy and continuity tiers

Redundancy is applied by data class and operational criticality. More replicas do not automatically create correctness: promotion, fencing, idempotency, backup independence and regularly exercised recovery are equally important.

Component	Baseline production redundancy	Failure behavior
Application / realtime	At least two nodes across availability zones behind health-aware routing	Drain failed node; reconnect with jitter; bounded replay from authoritative deltas
PostgreSQL authority	Managed HA or synchronous in-region standby; independent PITR backups	One fenced writer; automated or operator-approved promotion; no split brain
Regional disaster recovery	Asynchronous cross-region replica and copied backup/evidence catalog by tier	Promote a new single writer through a tested runbook; accept declared RPO, never pretend zero loss
Object evidence	Multi-zone durability, versioning and optional cross-region copy by retention class	Evidence pointers remain stable; missing copy raises an integrity incident
Signal telemetry	Replicated ingest/storage appropriate to retention and site buffer capacity	Edge continues buffering to quota; core uses prior validated observations
Edge	Durable site queue, redundant gateway for critical lines, local configuration and device failover	Plant continues authorized local work; cloud outage never removes certified machine safety
Link / Relay delivery	Durable outbox, multiple workers, endpoint/provider health and replay queue	At-least-once delivery with idempotency; provider failure does not block core commit
Control plane	Multi-zone routing and desired-state service with minimal domain data	Existing cells continue operating if control plane is unavailable

Continuity tiers

Tier	Example	Target posture
C0 - local continuity	Operator scanning, attendance capture	Client/edge queues and local projections continue through connectivity loss
C1 - high availability	Ordinary cell operation	Multi-zone application and database failover; RPO near zero in-region, RTO measured in minutes
C2 - regional recovery	Contractually critical plant or school group	Cross-region data/evidence copy; declared RPO and rehearsed RTO; controlled DNS/routing promotion
C3 - exceptional continuity	A specifically justified service	Additional regional capacity and manual/automated safeguards designed from its own consistency analysis

Why not global active-active relational writes by default

Manufacturing and education commitments require strong ordering and understandable conflict behavior. The baseline is a single fenced writer per cell with active-active stateless services, read delivery and edge continuity. Multi-writer regional authority is introduced only for a bounded domain whose conflict semantics have been proven; infrastructure optimism is not a conflict-resolution strategy.

Release and recovery exercises

- Kill an application node, availability zone dependency, queue worker and integration provider during production-shaped tests.
- Promote a PostgreSQL standby and prove fencing, connection recovery, idempotency and evidence continuity.
- Restore backups into an isolated verification cell and reconcile record counts, hashes, policies and object evidence.
- Run canary failure, feature-flag reversal and forward migration recovery before a high-risk release.
- Measure actual RPO, RTO and degraded user experience; update the continuity tier when evidence disagrees with the design.

Provisional deployment objectives

Operation	Initial engineering target	Clock excludes
Code-only ephemeral sandbox	p95 ready within 10 minutes after artifact is available	Large seed/restore preparation
Deploy existing digest to existing cell	p95 rollout within 5 minutes	Required canary observation window
Create production-shaped pre-production cell	Ready within 20 minutes	Masked dataset generation or large restore
Stop failed canary and drain traffic	Automated response within 2 minutes of a decisive gate	Human investigation
Destroy expired sandbox	Automatic within policy window with evidence retained	External provider cleanup outside control

PART 17

Technology decisions - consequences of the needs

The following stack is the recommended 2026 starting point, not the product identity. Each choice has a replacement boundary and a benchmark that can overturn it.

Need	Initial choice	Why	Replacement boundary
Realtime coordination	Elixir + Phoenix	Lightweight concurrency, fault isolation and realtime channels [S16]	Typed HTTP/websocket/capability contracts; cells can be reimplemented
Relational authority	PostgreSQL	Transactions, constraints, RLS, partitioning and logical replication [S17][S18][S19]	Domain export/event contracts and evidence, not generic repository portability
Web experience	TypeScript + React official SDK	Large talent/ecosystem and unrestricted custom interaction	Open HTTP/websocket/sync boundary; framework is not mandatory
Native mobile	PWA first; Flutter when native capability is required	One controlled operational shell with strong custom UI and device integration	Capability/sync contracts also support Swift, Kotlin or future clients
Local-first web	SQLite WASM + OPFS; Service Worker; Web Workers	Fast local projection and offline continuity [S22][S23][S24]	Projection/sync protocol supports native SQLite or future browser storage
Edge and portable compute	Rust + selective WebAssembly	Memory safety, small binaries, protocol integration and deterministic compute	Device/observation/command contracts
Optimization	Typed IR; CP-SAT, MIP/LP and specialized solvers	Problem classes differ; mature solvers beat a home-grown universal engine	Decision problem/result/explanation contract
Telemetry	ClickHouse candidate + object storage	Columnar high-volume time/event analytics; benchmark per site	Typed Signal ingest and observation-promotion contracts
Events/jobs	PostgreSQL outbox/inbox and durable Elixir jobs	Transactional simplicity first	Add NATS for proven fan-out; Temporal for proven long orchestration
Identity	External OIDC provider	Standards-based authentication and federation [S26]	OIDC claims map to Actualis principals; domain authorization remains owned
Evidence	S3-compatible object storage	Durability, lifecycle tiers, scale and ecosystem	Content-addressed evidence API and metadata in PostgreSQL
AI	One primary model initially through task contracts	Evaluation depth over premature multi-provider complexity	Model registry, typed outputs, tools, context packages and replay

Technology deliberately deferred

OpenSearch, a graph database, a vector database, NATS, Temporal, Kubernetes, multiple AI providers and a separate analytics platform are not forbidden. They enter only after a named quality scenario fails with the simpler architecture. Unlimited development budget does not remove cognitive cost.

DECISION RULE

A new platform component requires an owner, failure mode, SLO, security boundary, backup/recovery story, measured workload and documented exit path.

PART 17A

Integration and protocol architecture

Capabilities and events are canonical. HTTP, webhooks, JSON-RPC, brokers and future protocols are replaceable delivery adapters at the boundary.

Actualis Link is the system-to-system integration plane. It publishes a contract catalog, authenticates integration principals, translates supported wire protocols into governed capabilities or event subscriptions, applies quotas and records end-to-end delivery evidence. It never exposes tables or permits an adapter to bypass domain invariants.

Interaction patterns, not protocol religion

Need	Default boundary	Alternative adapters	Canonical meaning
Synchronous command/query	HTTPS + JSON with OpenAPI description [S35]	JSON-RPC 2.0, gRPC or future RPC	Invoke one versioned governed capability and return typed result/error
Realtime client delta	WebSocket; SSE where one-way streaming is sufficient	Future streaming transport	Subscribe to an authorized surface projection and resume from cursor
External event delivery	Signed webhook with CloudEvents-compatible envelope [S37]	Broker adapter described by AsyncAPI [S36]	Deliver a versioned fact after authoritative commit; never imply exactly-once
Bulk exchange	Signed object upload/download plus manifest and checksum	Managed transfer or customer object store	Import/export capability with staging, validation and reconciliation
Industrial boundary	MQTT / OPC UA through Actualis Edge	Vendor or field protocol driver	Typed observation or authorized command with device provenance
AI ecosystem	Internal typed tools; MCP adapter externally	Future agent/tool protocol	Same principal, purpose, policy, risk and evidence as every other caller

OpenAPI provides a language-agnostic HTTP interface description; AsyncAPI describes message-driven APIs without requiring one transport; JSON-RPC 2.0 is deliberately transport-agnostic [S35][S36][S38]. Actualis uses these standards as generated views of its contracts, not as the domain source.

Protocol adapter contract

Adapter must	Adapter must not
Authenticate a named integration/device principal and bind tenant, purpose, scopes and assurance	Invent a shared service account with unrestricted cross-tenant access
Validate protocol syntax, schema/version, size, encoding, rate and replay window before translation	Pass untrusted payloads directly to domain code or a database
Map one request to an explicit capability, query, subscription, upload or event-delivery contract	Expose generic table CRUD or let protocol method names become the domain model
Preserve correlation, idempotency, trace, source time and evidence through the translation	Promise ordering, exactly-once delivery or rollback that the transport cannot provide
Return stable error/reason codes and publish compatibility/deprecation information	Leak policy-protected facts through error details or schema discovery
Pass the same authorization and risk gates used by first-party experiences	Treat an internal network, webhook secret or API key as blanket authority

PART 17B

Webhooks, JSON-RPC and future adapters

Outbound webhooks

Stage	Behavior
Register	Verify endpoint ownership; bind event types, cell, purpose, schema versions, secret/key and delivery quota
Enqueue	Create delivery from transactional outbox only after authoritative commit; assign immutable event and attempt IDs
Sign	Cover payload digest, event ID and timestamp; support secret/key rotation and a bounded replay window
Deliver	Timeout, exponential backoff with jitter, concurrency cap and endpoint health/circuit state
Acknowledge	Treat only declared response classes as success; record latency, response class and endpoint version
Recover	Dead-letter after policy; provide filtered inspector, safe payload view, replay and redrive with new attempt ID
Evolve	Deliver declared schema version; inventory consumers; deprecate with telemetry and a compatibility window

Inbound webhooks

Inbound delivery terminates in a quarantine boundary: authenticate signature or mTLS identity, reject stale/replayed timestamps, enforce size and content type, validate schema, persist safe raw evidence, deduplicate, then translate to a proposed observation or governed command. It never writes authoritative tables directly. Unknown versions remain quarantined for inspection.

JSON-RPC 2.0

- A JSON-RPC method maps to a versioned Actualis capability or query; named parameters are preferred and validated against the canonical schema.
- Request ID is correlation, not business idempotency. Consequential commands still require an explicit idempotency key and expected version.
- Batch size, total cost and concurrency are bounded; each element is independently authorized and audited.
- JSON-RPC notifications have no response by definition [S38]; they are not accepted for consequential commands that require confirmation.

- Standard JSON-RPC errors wrap stable Actualis reason codes without revealing protected resource or policy details.

Future protocol rule

A new protocol adapter is accepted only if it can preserve principal identity, tenant/purpose, typed schema, correlation, idempotency, authorization, error semantics, trace/evidence and backpressure. If it cannot, the capability is not exposed through that protocol. The adapter passes a shared conformance suite and has an owner, threat model, quota, SLO and deprecation path.

Integration sandbox

- Issue environment-specific credentials that cannot address production and expire automatically.
- Provide webhook endpoint simulation, signature examples, request inspector, controlled redelivery and duplicate/out-of-order tests.
- Generate OpenAPI/AsyncAPI documents, JSON-RPC examples, SDKs and contract fixtures from the same capability/event version.
- Offer provider/device simulators, rate-limit behavior, timeout injection and deterministic failure cases.
- Promote an integration registration only after conformance, permission, replay, quota and failure-path tests pass.

Contract lifecycle and deprecation

State	Rule
Draft	Not routable outside sandbox; owner, threat model, schema and compatibility policy required
Published	Immutable version; generated documentation/fixtures; consumers and observed usage inventoried
Additive evolution	New optional fields or operations pass old/new consumer conformance before release
Breaking evolution	New major capability/event version; explicit translation only when semantics remain honest
Deprecated	Notice, migration guide, usage telemetry, target date and accountable exception process
Retired	Routing removed after policy; replay/evidence remains readable for its retention period

PART 18

Domain packages and product surfaces

Each product is assembled from a domain package, projections and one or more experience packages. Shared engines receive typed views; domain packages preserve meaning and invariants.

Product	Domain package owns	Shared Actualis capabilities
MES	Work orders, operations, routing execution, WIP, station state, quality gates, genealogy	Identity, capability, observation, dispatch, evidence, Edge, Relay
APS	Finite capacity, materials, changeovers, labor, maintenance and delivery objectives	Scenario, solver portfolio, commitment, approval, explanation
Task scheduling	Task semantics, skills, precedence, locations, service windows and SLA	Calendar, capability, constraints, scenario, dispatch, acknowledgement
Education operations	Students/cohorts, courses, teachers, rooms, support, attendance, safeguarding and publication	Commitments, schedules, authority, communication, evidence, Guide
Storefront	Catalog/offer semantics, configuration, basket and payment-provider handoff	Identity, availability/reservation, capability, public Surface, evidence
Website	Content, publication, forms and public navigation	Surface deployment, identity where needed, Relay, analytics projection

Domain source and generated infrastructure

A typed domain source or DSL may generate schemas, standard APIs, migration skeletons, events, policy hooks, basic admin forms, fixtures, tests and documentation. It must not generate the planner workbench, scanning flow, timetable conflict editor or quality station and pretend the product is finished.

Generate	Handcraft
Relational schema and explicit migration skeleton	Domain invariant implementation and failure semantics
Typed command/query/event clients	High-value workflows and decision interactions
Policy stubs and test matrices	Purpose/relationship decisions and threat cases
Admin CRUD and diagnostics	Role-specific experience packages
Solver IR adapters and fixtures	Mathematical formulation, heuristics and explanation
Observability, evidence and conformance hooks	Operational SLOs and incident runbooks

PART 19

Three reference journeys

Journey A - manufacturing exception and replan

- An authenticated RFID gateway observes a pallet movement. Raw evidence lands in Signal; normalization and validation promote a material-location observation into the manufacturing domain.
- A quality policy detects that the batch is on hold. The issue-to-production capability is denied; the operator surface immediately explains the allowed quarantine move.
- Affected production commitments are identified. A scenario snapshot is created with the hold as a hard constraint and delivery/service objectives preserved.
- Actualis Decide runs appropriate solvers and returns feasible alternatives with trade-offs. Actualis Guide explains consequences but does not create feasibility.
- A planner selects a plan; policy requires quality and production approval. After approval, commitments are versioned, work is dispatched to Edge and stakeholders receive targeted Relay messages.
- Execution acknowledgements and machine observations verify outcomes. Every transition links to the source observation, plan version, approvals, commands and evidence.

Journey B - teacher absence and timetable change

- A staff absence capability records the observation with privacy-limited detail. The system identifies published lessons and student support commitments at risk.
- Hard constraints include qualifications, safeguarding and room capacity. Soft objectives include schedule stability, learner disruption and workload fairness.
- The solver returns alternatives. The Challenge Engine notes that the lowest-disruption option overlaps two support sessions and asks whether wellbeing should outweigh timetable stability.
- An authorized scheduler selects and approves. The published commitment changes only then.
- The planner workbench shows dense conflicts; the teacher mobile view shows changed duties; the learner surface shows only relevant information; guardians receive messages according to school policy.
- Acknowledgements and attendance later provide outcome evidence without exposing staff medical detail.

Journey C - branded content and commerce

- A trusted development team deploys a fully custom public experience package with its own design system, animation and interaction model.
- The surface reads published offerings and availability through public capabilities and serves static/SSR pages through a CDN.
- A reservation enters the authoritative domain with idempotency and expiry. Payment is handed to a specialized provider; Actualis records the commitment and evidence.
- This proves that the core is headless and composable, but commerce concepts remain in their package rather than reshaping manufacturing or education.

PART 20

Implementation plan

0	Reality contracts Journeys, invariants, threats, UI prototypes, workload envelopes	0-8 weeks
1	Walking skeleton One cell, auth, capabilities, outbox, projection, evidence, edge simulator	2-5 months
2	Proof slices Manufacturing exception, education replanning, content/commerce surface	5-10 months
3	Operational hardening HA, DR, support access, load, security, deployment automation, SDK	10-16 months
4	Product expansion MES/APS packs, surfaces, telemetry scale and partner contracts	16+ months

Figure 2. Delivery phases are evidence gates, not a plan to build the entire platform before a user journey.

Phase 0 - reality contracts

- Shadow planners, operators, school schedulers, teachers, support staff and IT/security stakeholders.
- Write 20-30 operational narratives with normal flow, exception, offline case, authority, evidence, privacy and recovery need.
- Define the load envelope in measurable units and prepare production-shaped safe datasets.
- Prototype at least two radically different education surfaces and one scan-first manufacturing surface before fixing the SDK.
- Threat-model cells, support access, device enrollment, sync, AI tools and cross-system integrations.
- Benchmark PostgreSQL layout, Phoenix connections, SQLite/OPFS sync and ClickHouse telemetry ingest/query.

Phase 1 - the walking skeleton

Deliver one thin vertical path: OIDC login -> principal/purpose -> capability command -> PostgreSQL transaction -> outbox -> realtime delta -> local projection -> evidence -> Relay notification -> audit query. Add an edge simulator, one real reader and one signed webhook through Actualis Link. The same immutable artifact must promote through sandbox, test and pre-production to a canary cell, restore from backup and withstand reconnect, duplicate-delivery and load tests.

Phase 2 - three proof slices

Implement the journeys from Part 19. Manufacturing proves device trust and operational evidence; education proves constraints, privacy, concurrency and distinct surfaces; content/commerce proves headless UI freedom and public performance.

Phase 3 - hardening before expansion

Stabilize capability/event compatibility, security review, support controls, disaster recovery, cell operations, policy tests, observability, performance budgets and SDK only after the proof slices reveal genuine repetition.

STOP RULE

If the third product slice needs a new universal entity hierarchy, pause. First ask whether the capability contract is too weak or the domain package is incorrectly bounded. Do not expand the ontology reflexively.

PART 21

Repository, engineering workflow and CI

Suggested repository shape

```
/architecture ADRs, quality scenarios, threat models
/contracts capability, event, sync and solver schemas
/kernel authority, commitments, execution, evidence
/modules manufacturing, education, commerce packages
/projections surface and analytical projections
/experiences independent web/mobile/public packages
/edge gateway, protocol drivers and simulator
/link API, webhook, RPC adapters and integration sandbox
/compute solver, simulation and AI workers
/sdk generated clients and experience toolkit
/evals solver fixtures, AI replay, policy/data tests
/deployment cell templates, migrations, runbooks and SLOs
```

Mandatory pipeline gates

Gate	Proof required
Contract	Backward/forward compatibility, generated clients and consumer conformance
Migration	Production-shaped rehearsal, lock/time budget, rollback/forward recovery and evidence
Policy	Principal x relationship x purpose x data-class matrix; cross-cell and support negative tests
Domain	Invariant, idempotency, concurrency, effective-date and reconciliation tests
Experience	Visual regression, accessibility, offline/reconnect, local-data scope and performance budget
Performance	Connection/fan-out, commands, 1 TB query plans, device storm, solver/AI isolation
Security	SAST/dependency/secret scan, SBOM, signed artifacts, threat regression and penetration tests
Integration	OpenAPI/AsyncAPI/JSON-RPC conformance, webhook signature/replay, idempotency, quota and compatibility
Deployment	Ephemeral sandbox create/destroy, immutable promotion, canary health, migration and rollback/roll-forward exercise
AI	Grounding, metric correctness, policy, tool selection, leakage, prompt injection and replay outcome
Operations	Backup restore, cell failover, telemetry loss, queue backlog and provider-outage exercises

Team ownership

Organize around enduring responsibilities, not language silos: Core Authority, Domain Manufacturing, Domain Education, Surface, Edge/Signal, Decide, Guide and Platform Reliability. Every capability has one domain owner; every data class and SLO has an operational owner. Cross-team decisions live as ADRs and executable conformance tests.

PART 22

Architecture decision records

ADR	Decision	Status / consequence
0001	Capability-centric modular monolith with server-authoritative, local-first clients	Accepted baseline. Extract only independently scaling or security-sensitive workloads.
0002	Experience packages are first-class and independently deployable	Accepted baseline. Official SDK is optional; open headless contracts are mandatory.
0003	Data-class ownership over database neutrality	Accepted baseline. PostgreSQL owns relational truth; telemetry, evidence and projections have explicit owners.
0004	Cell is the unit of data placement, scaling and recovery	Accepted baseline. Cross-cell operations are explicit; control plane minimizes business data.
0005	Separate observation, inference, decision and command	Accepted baseline. Consequential decisions reference captured validated observations and evidence.
0006	AI is a governed principal, never authority or source of truth	Accepted baseline. Typed tools, context policy, validation, replay and human/deterministic approval.
0007	Solver portfolio behind a typed decision contract	Accepted baseline. Rust orchestrates; mature solvers solve; explanation is part of result.
0008	Selective temporality	Accepted baseline. Ordinary, effective-dated and fully bitemporal records chosen per invariant.
0009	Fine-grained authorization is a first-class framework capability	Accepted baseline. Policies combine capability, relationship, attributes, purpose, context, risk, fields and obligations.
0010	PostgreSQL outbox and native durable execution first	Proposed. NATS/Temporal only after measured distributed or long workflow need.
0011	ClickHouse as default raw-telemetry candidate	Proposed pending site-shaped benchmark; no raw telemetry in command database.
0012	Immutable artifacts promote through template-created environments	Accepted baseline. Sandbox, test, pre-production and production use one signed digest and explicit configuration.
0013	Redundancy is tiered; each cell has one fenced relational writer	Accepted baseline. Multi-zone HA, backups and optional cross-region DR; no default global multi-writer authority.
0014	Wire protocols are adapters around canonical capability and event contracts	Accepted baseline. Link supports HTTP, webhooks, JSON-RPC and future adapters through one conformance boundary.

Selective temporality decision

Persistence level	Use
Ordinary operational	Current state plus audit for records without effective-date reconstruction need
Effective-dated	Published schedules, qualifications, prices, contracts, policies and planned configuration
Fully bitemporal	Only when real-world validity and historical system belief must be reconstructed independently

PART 23

Risks, countermeasures and kill criteria

Risk	Early signal	Countermeasure / kill criterion
Platform before product	Months of kernel work without a complete operator or scheduler journey	Walking skeleton plus proof slices; stop abstractions without two real consumers
Universal ontology	Generic entity/attribute tables and debates over one hierarchy	Domain relational schemas; shared semantic interfaces only
Polyglot sprawl	Teams debug five runtimes for an ordinary command	Elixir/PostgreSQL core; Rust only at edge/compute; add infrastructure by ADR
Sync complexity	Invisible conflicts, oversized projections, unreliable offline replay	Surface-scoped projections, explicit offline policy, deterministic reconciliation and chaos tests
UI fragmentation	Each surface rebuilds security, sync and accessibility incorrectly	Small conformance SDK and common contracts; do not force one design system
Client data exposure	Broad caches or stale sensitive data after role change	Purpose-scoped projection, encryption/expiry, revocation and minimum fields
Solver abstraction too neutral	IR cannot express domain propagation or explanation	Domain extensions and specialized adapters rather than bloating the kernel
AI noise/overreach	Many unacted insights, weak citations, users defer to model	Attention ranking, evidence cards, uncertainty, bounded tools and outcome evaluation
Device trust failure	Shared credentials, replay, unknown firmware and clock drift	Per-device identity, sequence/evidence, attestation, quarantine and site runbooks
Cell operations burden	Bespoke deployments and uncontrolled version skew	Cell templates, fleet control, compatibility windows and dedicated-cell thresholds

Decisions still open

- Which first product and customer journey owns the roadmap: manufacturing exception/replan or education timetable adaptation?
- What is the exact concurrency and telemetry envelope per cell and per site?
- Which regulatory regions and data residency commitments are in launch scope?
- How much offline autonomy may each capability grant to an edge or client principal?
- Which first solver families and explanation standards are required by the launch problem?
- Which identity provider, notification channels and commerce boundary are launch dependencies?
- What legal retention and evidence requirements apply to the first customer contracts?
- Does the working name Actualis clear legal and domain review?

PART 24

First 90 days

Weeks	Outcome	Deliverables
1-2	Choose the lead operational decision	Named problem owner; journey; actors; failure modes; decision rights; baseline metrics
2-4	Freeze reality contracts v0	Capability template; observation/command envelope; classification; ADRs; threat model
3-6	Prove experience freedom	Two education surfaces or planner/operator pair using mocked typed contracts; local projection
4-8	Prove physical assumptions	1 TB PostgreSQL benchmark; 10k connection/fan-out; telemetry; edge offline/replay
6-10	Build walking skeleton	One cell promoted sandbox-to-canary; OIDC; command; outbox; sync; Link/Relay; restore and load tests
10-12	Commit build plan	Validated bottlenecks; updated ADRs; slice backlog; SLO/error budgets; team plan

Definition of success at day 90

- A stakeholder completes one real governed action end to end and sees why it was allowed or denied.
- One signed artifact is promoted from an ephemeral sandbox through pre-production to a canary, with migration and recovery evidence.
- The same domain capability is consumed by two visibly and behaviorally different experience packages.
- A simulated device disconnects, buffers, reconnects and promotes one observation without duplication.
- A partner simulator proves signed webhook delivery, duplicate/replay handling and one JSON-RPC adapter over the same capability.
- A cell survives the provisional connection test, restores from backup and demonstrates workload isolation.
- Production-shaped measurements confirm or replace the initial technology hypotheses.
- No component exists without an owner, failure mode, SLO and reason grounded in a quality scenario.

START HERE

Do not begin by implementing a generic schema compiler. Begin with one consequential decision, two distinct surfaces, one machine observation and one complete evidence trail. Extract the compiler from repetition that actually appears.

PART A

Appendix - quality scenarios

ID	Stimulus	Expected response
QS-01	10,000 clients reconnect after a 5-minute regional network interruption	Jittered reconnect, bounded snapshot/delta load, no pool exhaustion; commands retain SLO
QS-02	RFID gateway resends 24 hours with duplicate sequence numbers	Raw evidence linked once; validated observations idempotent; ordering uncertainty visible
QS-03	A teacher's relationship changes while their device is offline	New sync denied, stale projection expires, queued commands re-authorized at commit
QS-04	Solver workers use all allocated CPU during an urgent operator scan	Scan command and realtime acknowledgement stay within SLO; solver throttled/cancelled
QS-05	An untrusted document instructs AI to reveal hidden student data	Content is evidence, not instruction; authority cannot expand; event is logged
QS-06	Planner publishes while another scenario is selected	Expected-version check rejects stale commit; no silent overwrite
QS-07	Telemetry store is unavailable	Core continues; edge buffers to quota; validated observations remain; recovery explicit
QS-08	Provider accepts but never delivers a school closure message	Relay tracks state, changes channel/escalates and records unresolved recipients
QS-09	Support needs urgent access to a sensitive cell	JIT scoped request, approval, recorded session, expiry and post-access review
QS-10	New experience targets an older capability version	Compatibility checked before routing; unsupported package blocked or falls back
QS-11	Migration runs on production-shaped 1 TB estate	Lock and replication lag remain within budget; rolling deployment remains possible
QS-12	AI provider is unavailable for 12 hours	Scheduling, scanning, approvals, communications and evidence continue; AI degrades clearly
QS-13	A substitute teacher is assigned for one day	Access is limited to assigned classes, required fields and teaching purpose; it expires automatically
QS-14	An administrator broadens a policy affecting 4,000 learners	Policy Studio shows impact and sensitive-field expansion; peer approval and effective date are required
QS-15	Emergency support requests break-glass access	Step-up, scoped duration, reason, recording, notification and mandatory after-action review apply
QS-16	A planner, export worker and AI request the same employee record	Each receives the permitted projection and obligations for its purpose; restricted fields never leave the cell
QS-17	A branch requests an ephemeral sandbox	Isolated routable cell with synthetic seed, test identities, observability and expiry is ready within the declared time budget
QS-18	A canary release increases command errors	Automated gate stops promotion, drains canary traffic and preserves evidence; compatible rollback or roll-forward executes
QS-19	The active application zone fails	Healthy nodes retain command SLO; realtime clients reconnect with jitter; no duplicate authoritative effects
QS-20	The primary region is lost	Declared tier runbook promotes a fenced writer; measured RPO/RTO and any loss are reported and reconciled
QS-21	Partner sends duplicate, stale and out-of-order webhooks	Signature/time/schema validation, replay protection and idempotency prevent duplicate or unauthorized effects
QS-22	A new transport adapter invokes an existing capability	Shared conformance proves identical identity, authorization, schema, errors, idempotency, evidence and backpressure

PART B

Appendix - example capability contract

```
capability: manufacturing.material.move
version: 1
command:
  pallet_id: PalletId
  from_location: LocationId
  to_location: LocationId
  expected_version: Integer
principal:
  types: [human, edge_gateway]
  purpose: execute_material_flow
  authority:
    relation: assigned_to_site
  invariants:
    - pallet.current_location == from_location
    - destination.accepts(material_class)
    - quality_status permits movement
  delivery:
    idempotency_key: required
    offline: quarantine_move_only
    expires_after: 15_minutes
  effects:
    - update pallet location
    - emit material.moved.v1
    - update affected commitments
  evidence:
    - source observation or human confirmation
    - policy and capability versions
    - principal, device, timestamps and result
  ai:
    read: allowed_if_authorized
    propose: allowed
    execute: forbidden
```

Contract commentary

The example is illustrative, not a final DSL. Its value is the questions it forces: exact actor and purpose, preconditions, offline authority, expiry, version conflict, evidence and AI policy. Compilation may generate validation, API schema, fixtures, audit hooks, client types and diagnostics. Domain code still implements the invariant and user experience.

PART C

Appendix - evaluation scorecards

Architecture fitness scorecard

Dimension	Evidence	Release threshold
Operational fit	Journey completion, exception handling, planner/operator feedback	Named owner accepts normal and failure flows
Authority	Policy matrix and denied-action evidence	No unclassified consequential capability
Experience	Two distinct surfaces; accessibility and local performance	No core dependency on one UI renderer
Data	Ownership, lineage, retention, migration and restore	Every data class has owner and recovery path
Scale	Production-shaped load/fan-out/device tests	SLOs hold under lane saturation and reconnect
Security/privacy	Threat tests, review and incident exercises	No critical finding; high risks explicitly accepted
Operability	SLO dashboards, trace/evidence, runbooks and cost metrics	On-call can diagnose and restore a cell
Replaceability	Contract conformance and simulated adapter replacement	No hidden cross-store or cross-module access

AI evaluation scorecard

Dimension	Test
Grounding	Every factual claim maps to permitted evidence; unsupported claims are flagged
Metric correctness	Metric identity/version, filters, time basis and calculation match semantics
Authority	Agent cannot broaden context, call ungranted tools, self-approve or bypass expected version
Decision quality	Recognizes hard constraints, affected commitments, stakeholders and reversibility
Calibration	Uncertainty and alternatives match evidence; correlation is not called causation
Challenge value	Raises relevant blind spots without noise; users can act, defer or reject
Privacy/security	No cross-cell/purpose leakage; resistant to prompt injection and malicious content
Reproducibility	Model, prompt, context, retrieval, tools, validators and output can be replayed
Outcome	Track acceptance, correction, operational result and interruption cost

NIST's AI Risk Management Framework generative-AI profile supports treating trustworthiness across design, deployment, use and evaluation rather than as a final filter [S29]. OpenTelemetry's GenAI conventions provide useful evolving observability semantics [S32].

PART D

Appendix - glossary

Term	Meaning in Actualis
Capability	Governed typed command or query with authority, invariants, effects, evidence, risk and delivery semantics
Cell	Unit of business-data placement, scale, failure containment, security keys and recovery
Commitment	Authorized obligation or reservation distinct from a proposal, forecast or candidate plan
Data class	A category with one owner selected by consistency, access, retention and failure needs
Domain package	Specialized schemas, invariants, capabilities and semantics for a bounded domain
Evidence	Durable material supporting reconstruction of an observation, decision, command or outcome
Experience package	Independent product surface with its own interaction, projection, performance and release lifecycle
Integration adapter	Boundary component translating one wire protocol into canonical capability, query, event or subscription contracts
Immutable promotion	Moving the same signed artifact digest through environments while configuration and data remain separately governed
Insight	Versioned claim with evidence, uncertainty, alternatives, impact, provenance and review state
Observation	Recorded claim about reality with source, event/ingest time, quality and provenance
Obligation	A condition attached to authorization, such as masking, approval, logging, expiry, rate limit or notification
Permission policy	Versioned rule deciding capability, scope, fields and obligations from principal, relationship, purpose and context
Projection	Disposable, purpose-shaped read model derived from authoritative data or events
Principal	Human, device, integration, workload or AI identity that may invoke capabilities
Scenario	Versioned possible world and assumptions used to evaluate a decision
Validated observation	Observation accepted into a domain after explicit quality and context checks
Webhook	Signed at-least-once HTTP event delivery with explicit retries, idempotency, replay protection and observable attempts

PART E

Appendix - sources and standards

Sources were accessed in July 2026. Product documentation supports architectural observations; it does not imply that every deployment exhibits the same trade-offs.

ID	Source
S1	SAP - Extend SAP S/4HANA Cloud the Right Way https://news.sap.com/2025/08/extend-sap-s4hana-cloud-right-way-clean-clear/
S2	SAP Learning - Introducing the Clean Core Approach https://learning.sap.com/courses/practicing-clean-core-extensibility-for-sap-s-4hana-cloud/introducing-the-clean-core-approach_fcb6c662-7041-4c99-88bd-345636fae7f3
S3	Odoo 19 - ORM API https://www.odoo.com/documentation/19.0/developer/reference/backend/orm.html
S4	Odoo 19 - Upgrade scripts https://www.odoo.com/documentation/19.0/developer/reference/upgrades/upgrade_scripts.html
S5	Odoo 19 - Module manifests https://www.odoo.com/documentation/19.0/developer/reference/backend/module.html
S6	Frappe - DocTypes https://docs.frappe.io/framework/user/en/basics/doctypes
S7	Frappe - Framework basics https://docs.frappe.io/framework/user/en/basics
S8	Frappe - REST API https://docs.frappe.io/framework/user/en/guides/integration/rest_api
S9	Framework M - Introduction https://www.frameworkm.dev/docs/intro/
S10	Framework M - Architecture constraints https://www.frameworkm.dev/docs/architecture/architecture_constraints/
S11	Framework M - Architecture concepts https://www.frameworkm.dev/docs/architecture/concepts/
S12	PowerSchool - Notice of United States data breach https://www.powerschool.com/security/sis-incident/notice-of-united-states-data-breach/
S13	ManageBac - Enabling Public API https://help.managebac.com/hc/en-us/articles/360018226931-Enabling-ManageBac-Public-API-for-Integrations
S14	Faria Education - ManageBac REST API overview https://guide.fariaedu.com/integrations-portal/managebac/public-rest-apis/overview
S15	Instructure - Canvas APIs https://developerdocs.instructure.com/services/canvas
S16	Phoenix - Channels https://hexdocs.pm/phoenix/channels.html
S17	PostgreSQL - Row security policies https://www.postgresql.org/docs/current/ddl-rowsecurity.html
S18	PostgreSQL - Table partitioning https://www.postgresql.org/docs/current/ddl-partitioning.html
S19	PostgreSQL - Logical replication https://www.postgresql.org/docs/current/logical-replication.html
S20	OASIS - MQTT Version 5.0 https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html
S21	OPC Foundation - OPC UA https://opcfoundation.org/about/opc-technologies/opc-ua/

ID	Source
S22	SQLite - WASM persistence and OPFS https://sqlite.org/wasm/doc/trunk/persistence.md
S23	MDN - Service Worker API https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API
S24	MDN - Web Workers https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers
S25	NIST SP 800-207 - Zero Trust Architecture https://csrc.nist.gov/pubs/sp/800/207/final
S26	OpenID Foundation - OpenID Connect Core 1.0 https://openid.net/specs/openid-connect-core-1_0.html
S27	European Commission - Data protection explained https://commission.europa.eu/law/law-topic/data-protection/data-protection-explained_en
S28	European Commission - GDPR processing principles https://commission.europa.eu/law/law-topic/data-protection/rules-business-and-organisations/principles-gdpr/overview-principles/what-data-can-we-process-and-under-which-conditions_en
S29	NIST - AI RMF Generative AI Profile https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence
S30	MDN - Content Security Policy https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP
S31	W3C - PROV-O: The PROV Ontology https://www.w3.org/TR/prov-o/
S32	OpenTelemetry - GenAI observability https://opentelemetry.io/blog/2026/genai-observability/
S33	Model Context Protocol - Specification https://modelcontextprotocol.io/specification/2025-11-25
S34	W3C - CSS Custom Properties https://www.w3.org/TR/css-variables-1/
S35	OpenAPI Initiative - OpenAPI Specification 3.2.0 https://spec.openapis.org/oas/v3.2.0.html
S36	AsyncAPI Initiative - AsyncAPI Specification 3.0.0 https://www.asyncapi.com/docs/reference/specification/v3.0.0
S37	CNCF - CloudEvents https://cloudevents.io/
S38	JSON-RPC Working Group - JSON-RPC 2.0 Specification https://www.jsonrpc.org/specification

PART 25

Conclusion

The moat is not the number of models Actualis supports. It is the quality with which it turns reality into governed, explainable and executable commitments.

Actualis begins from manufacturing and education because those domains force the hard questions: What is true? What is only inferred? What has been promised? Who may change it? Which constraints are hard? Whose interests are missing? Can the decision still execute when the network or AI is unavailable? What evidence will reconstruct it years later?

The proposed architecture answers those questions with a capability-centric modular core, specialized domain schemas, independent experiences, cells, explicit data classes, an edge trust boundary, a solver portfolio, governed communication and AI that makes people better at asking questions without becoming authority.

The next step is deliberately small: one consequential decision, two different surfaces, one device observation, one authorized command and one complete evidence trail. If that walking skeleton works under realistic load, privacy, offline and recovery conditions, Actualis has a foundation worth extending. If it does not, reality has given us the information needed to change before the platform hardens.

FINAL RECOMMENDATION

Adopt the accepted ADRs in Part 22 as the architecture baseline. Begin Phase 0 immediately. Treat all named technologies as benchmarkable hypotheses, and do not stabilize the SDK, DSL or service split until the three proof slices demonstrate real reuse.